Q1. First Palindromic String

Aim: To find the first palindromic string in a list.

Algorithm:

1. Take input list of words.
2. Traverse each word.
3. Check if word == reverse of word.
4. Print first palindrome found, else print "

Code (Python):

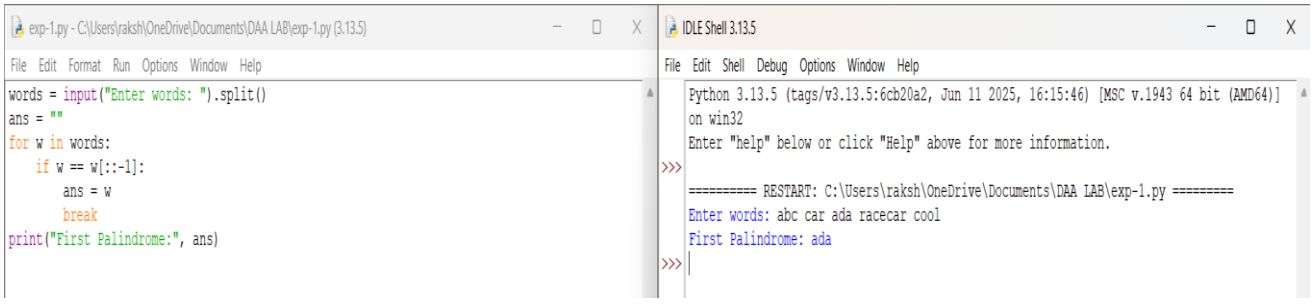
```
words = input("Enter words: ").split()
ans = ""
for w in words:
    if w == w[::-1]:
        ans = w
        break
print("First Palindrome:", ans)
```

Input:

abc car ada racecar cool

Output:

First Palindrome: ada



Time Complexity: O(n × m)

Space Complexity: O(m)

Result: The first palindromic string is found successfully.

Q2. Count Common Indices in Two Arrays

Aim: To count how many elements of one list exist in another.

Algorithm:

1. Read the two arrays.
2. Convert both arrays into sets.
3. Check each element of the first array in the second set.
4. Check each element of the second array in the first set.
5. Count and display the results.

Code (Python):

```
nums1 = list(map(int, input("Enter nums1: ").split()))
nums2 = list(map(int, input("Enter nums2: ").split()))
set1, set2 = set(nums1), set(nums2)
answer1 = sum(1 for x in nums1 if x in set2)
answer2 = sum(1 for x in nums2 if x in set1)
print("Output:", [answer1, answer2])
```

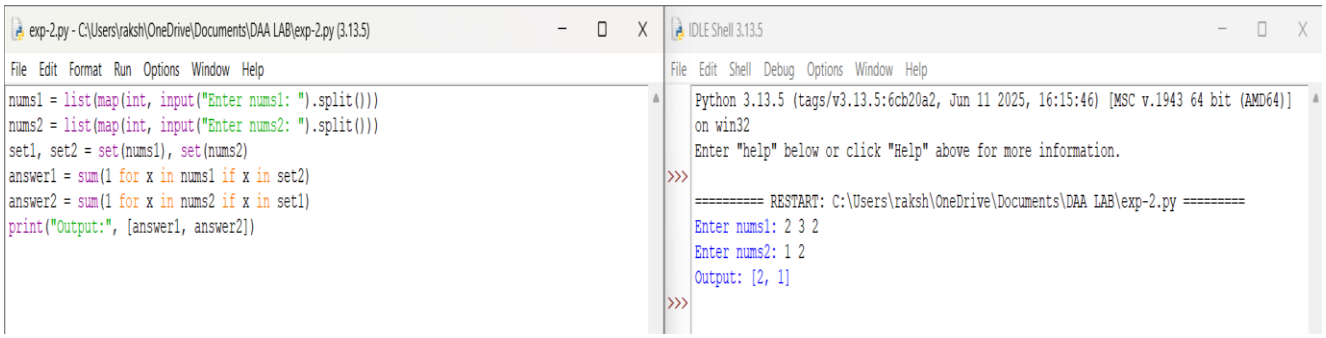
Input:

Enter nums1: 2 3 2

Enter nums2: 1 2

Output:

[2,1]



Time Complexity: O(n+m)

Space Complexity: O(n+m)

Result: The number of common elements in both arrays is counted successfully.

Q3. Sum of Squares of Distinct Counts of Subarrays

Aim: To calculate sum of (distinct_count²) for all subarrays.

Algorithm:

1. Read the array.
2. Start from each index.
3. Create an empty set for distinct elements.
4. Extend the subarray one element at a time.
5. Add each new element to the set.
6. Calculate the square of the number of distinct elements.
7. Add it to the total.
8. Display the total.

Code (Python):

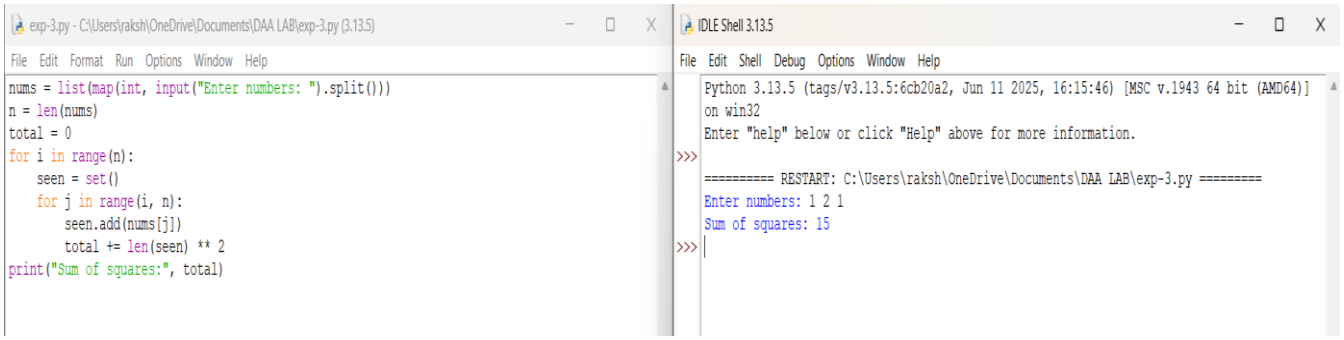
```
nums = list(map(int, input("Enter numbers: ").split()))
n = len(nums)
total = 0
for i in range(n):
    seen = set()
    for j in range(i, n):
        seen.add(nums[j])
        total += len(seen) ** 2
print("Sum of squares:", total)
```

Input:

Enter array: 1 2 1

Output:

Result: 15



Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

Result: The sum of squares of distinct counts for all subarrays is calculated successfully.

Q4. Count Pairs with Condition

Aim: To count pairs (i, j) where nums[i] == nums[j] and (i*j) divisible by k.

Algorithm:

1. Read the array and value of k.
2. Consider every possible pair (i, j).
3. Check whether the two elements are equal.
4. Check whether (i × j) is divisible by k.
5. If both conditions are true, increment the count.
6. Display the count.

Code (Python):

```
nums = list(map(int, input("Enter numbers: ").split()))
k = int(input("Enter k: "))
count = 0
n = len(nums)
for i in range(n):
    for j in range(i+1, n):
        if nums[i] == nums[j] and (i*j) % k == 0:
            count += 1
print("Pairs count:", count)
```

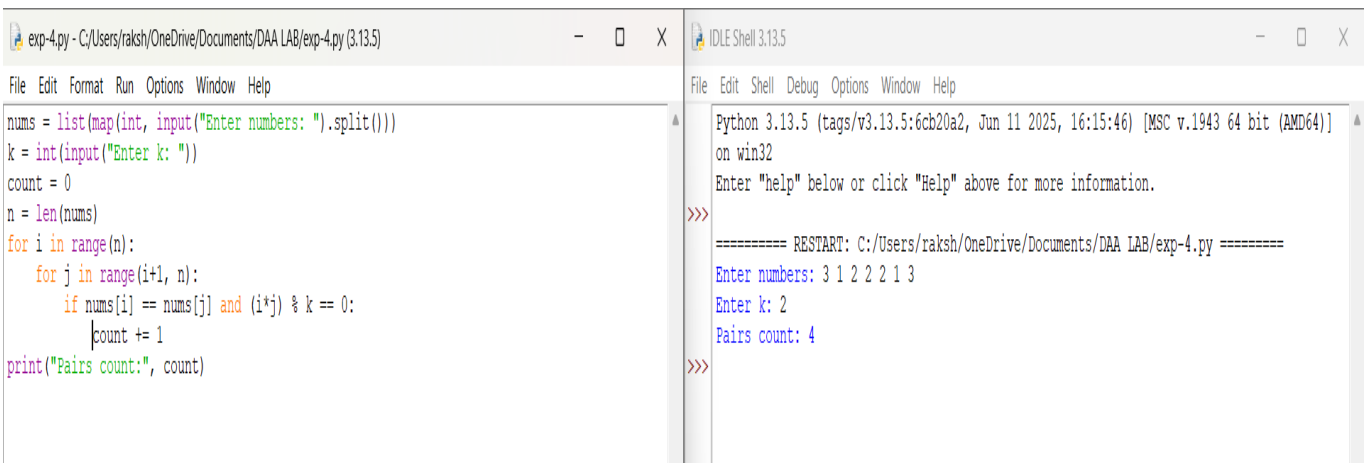
Input:

Enter nums: 3 1 2 2 2 1 3

Enter k: 2

Output:

Pairs: 4



Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Result: The number of valid pairs satisfying the given condition is counted successfully.

Q5. Max of Array

Aim: To find maximum element in array with least time complexity.

Algorithm:

1. Read the array.
2. Assume the first element is maximum.
3. Compare it with every remaining element.
4. If a larger element is found, update maximum.
5. Display the maximum element.

Code (Python):

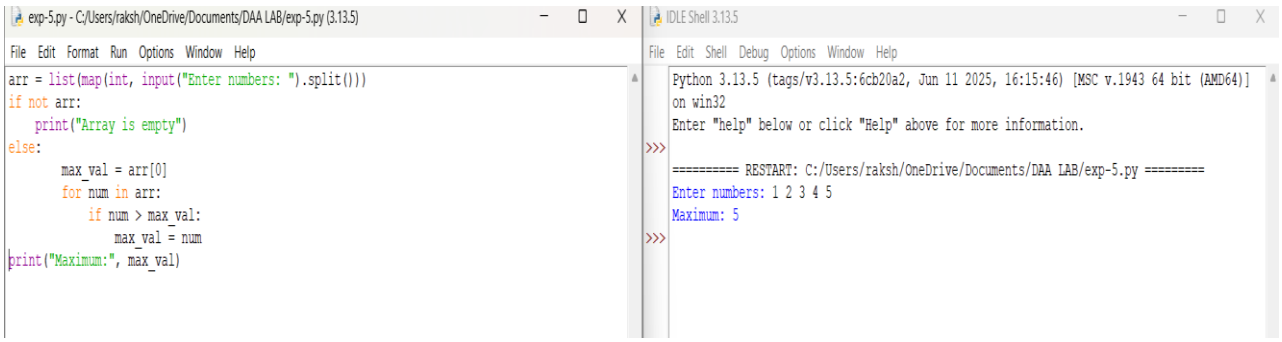
```
arr = list(map(int, input("Enter numbers: ").split()))
if not arr:
    print("Array is empty")
else:
    max_val = arr[0]
    for num in arr:
        if num > max_val:
            max_val = num
    print("Maximum:", max_val)
```

Input:

Enter array: 1 2 3 4 5

Output:

Max: 5



Time Complexity: O(n)

Space Complexity: O(1)

Result: The maximum element is found successfully.

Q6. Sort and Find Maximum

Aim: Sort array and return maximum element.

Algorithm:

1. Read the array.
2. Sort the array in ascending order.
3. Select the last element.
4. Display it as the maximum element.

Python Code:

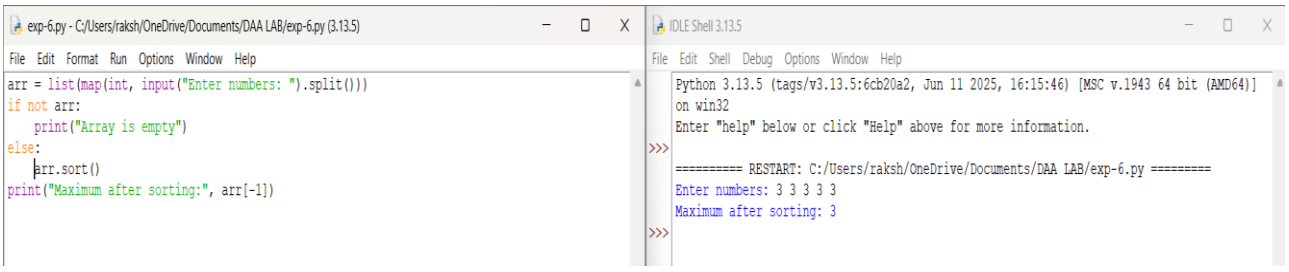
```
arr = list(map(int, input("Enter numbers: ").split()))
if not arr:
    print("Array is empty")
else:
    arr.sort()
    print("Maximum after sorting:", arr[-1])
```

Input:

Enter numbers:[3, 3, 3, 3, 3]

Output:

Max: 3



Time Complexity: O(n log n)

Space Complexity: O(1)

Result: The array is sorted and the maximum element is found successfully.

Q7. Unique Elements of List

Aim: Remove duplicates and return only unique elements.

Algorithm:

1. Read the array.
2. Convert the array into a set.
3. Sets automatically remove duplicate elements.
4. Convert the set back into a list.
5. Display the unique elements.

Python Code:

```
arr = list(map(int, input("Enter numbers: ").split()))

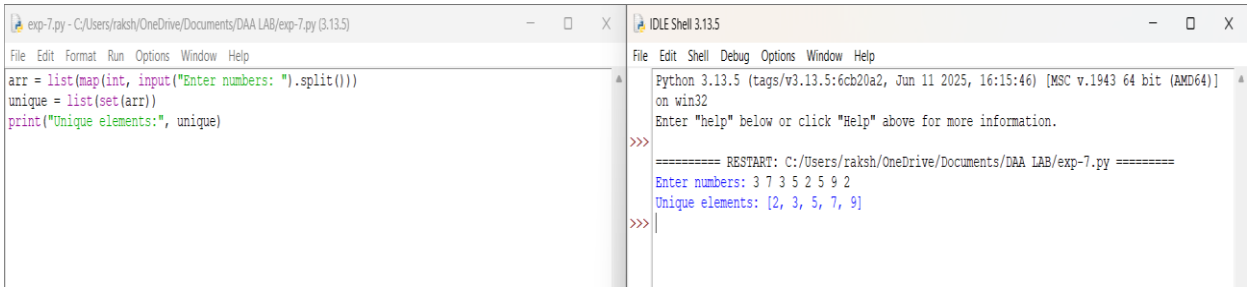
unique = list(set(arr))

print("Unique elements:", unique)
```

Input: 3 7 3 5 2 5 9 2

Output:

Unique: [2, 3, 5, 7, 9]



Time Complexity: O(n)

Space Complexity: O(n)

Result: Duplicate elements are removed and unique elements are obtained successfully.

Q8. Bubble Sort

Aim: Sort an array of integers using the Bubble Sort technique.

Algorithm:

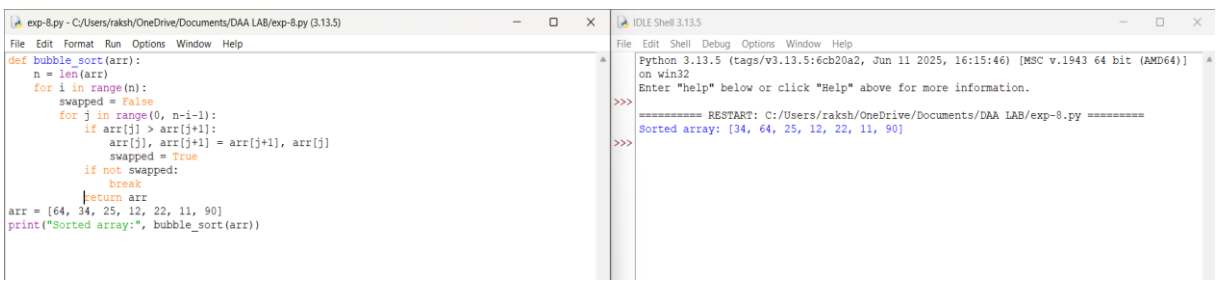
1. Read the array.
2. Compare adjacent elements.
3. Swap them if they are in the wrong order.
4. Repeat the process for all elements.
5. Stop early if no swapping occurs.
6. Display the sorted array.

Python Code:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
arr = [64, 34, 25, 12, 22, 11, 90]
print("Sorted array:", bubble_sort(arr))
```

Input: [64, 34, 25, 12, 22, 11, 90]

Output: [11, 12, 22, 25, 34, 64, 90]



Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Result: The array is sorted successfully using Bubble Sort.

Q9.Binary Search

Aim: Check if a given number exists in a sorted array using Binary Search.

Algorithm:

1. Read the array and search key.
2. Sort the array.
3. Set low to the first position and high to the last position.
4. Find the middle element.
5. If the middle element equals the key, return its position.
6. If the key is larger, search the right half.
7. Otherwise, search the left half.
8. Repeat until the element is found or the range becomes empty.

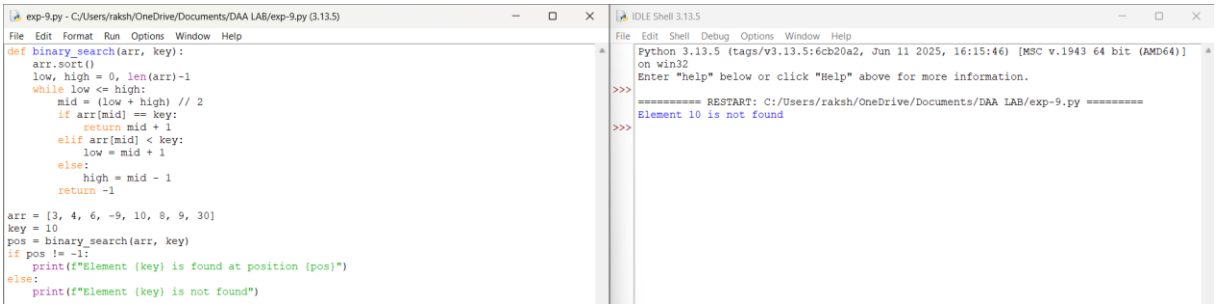
Python Code:

```
def binary_search(arr, key):
    arr.sort()
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid + 1
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
        return -1
arr = [3, 4, 6, -9, 10, 8, 9, 30]
key = 10
pos = binary_search(arr, key)
if pos != -1:
    print(f"Element {key} is found at position {pos}")
else:
    print(f"Element {key} is not found")
```

Input: Array = [3, 4, 6, -9, 10, 8, 9, 30], Key = 10

Output:

Element 10 is found at position 6



Time Complexity: O(n log n)

Space Complexity: O(1)

Result: The given element is successfully found using Binary Search.

Q10. Merge Sort (O(n log n) Sorting)

Aim: Sort an array of integers in ascending order using Merge Sort without built-in functions.

Algorithm:

1. Divide the array into two halves.
2. Recursively divide each half until single elements remain.
3. Compare elements of the two halves.
4. Merge them in sorted order.
5. Continue merging until the complete array is sorted.
6. Display the sorted array.

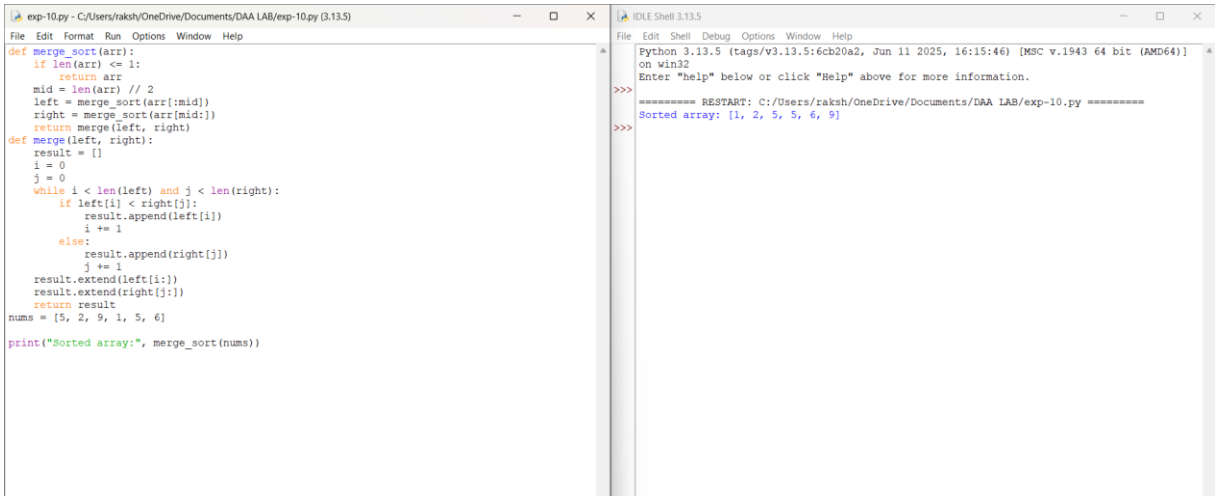
Code:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
nums = [5, 2, 9, 1, 5, 6]
```

print("Sorted array:", merge_sort(nums))

Input:[5, 2, 9, 1, 5, 6]

Output:[1, 2, 5, 5, 6, 9]



Time Complexity: O(n log n)

Space Complexity: O(n)

Result: The array is successfully sorted in ascending order using Merge Sort.